

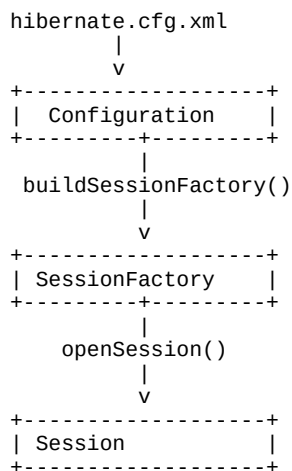
Chapter 3 - SessionFactory, Session & Transaction

1. SessionFactory

SessionFactory is the factory that creates Session objects. It is heavyweight, thread-safe, usually created once during application startup, and reused throughout the application's lifetime.

2. Internal Flow

A Configuration object reads Hibernate settings and entity mappings. It builds a SessionFactory, which opens Session objects.

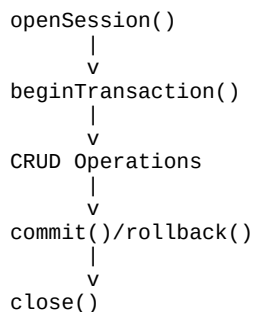


3. Session

Session is a lightweight, non-thread-safe object representing the Persistence Context. It manages entity objects, tracks changes (dirty checking), and contains the Level 1 Cache.

4. Session Lifecycle

Open Session → Begin Transaction → Perform CRUD Operations → Commit/Rollback → Close Session.



5. openSession() vs getCurrentSession()

openSession(): Always creates a new Session. You must close it manually.

getCurrentSession(): Returns the Session bound to the current context/transaction. Its lifecycle is

generally managed by the framework (for example, Spring).

6. Transaction

A Transaction groups database operations into a single unit of work and ensures ACID properties. Use `commit()` to persist changes and `rollback()` if an error occurs.

7. Sample Code

Typical Hibernate transaction flow:

```
Session session = sessionFactory.openSession();

Transaction tx = session.beginTransaction();

Employee emp = new Employee();
session.persist(emp);

tx.commit();
session.close();
```

8. Session vs SessionFactory

SessionFactory: Heavyweight, thread-safe, one per database/application.

Session: Lightweight, not thread-safe, one per request/transaction, contains Level 1 Cache.

9. Best Practices

- Create one SessionFactory per database.
- Never share a Session across threads.
- Always close Sessions.
- Keep transactions short.
- Perform database modifications inside transactions.

10. Interview Questions

1. Why is SessionFactory heavyweight? 2. Why is Session not thread-safe? 3. Difference between `openSession()` and `getCurrentSession()`? 4. Where is Level 1 Cache stored? 5. Why should every update be inside a transaction?